

Research Project
On
**Survey on Rust Microcontroller
Programming**

Submitted in partial fulfilment
of

Embedded Control

Summer Semester 2026



Under the Guidance of :

Prof. Dr. Lothar Berger

Submitted By :

Aditya Grewal (29541041)

**MASTER OF ELECTRICAL ENGINEERING AND
EMBEDDED SYSTEMS**

HOCHSCHULE RAVENSBURG-WEINGARTEN

September 5, 2026

Abstract

Rust promises memory and concurrency safety at compile time without a runtime or garbage collector, which makes it an attractive candidate for microcontroller firmware.

This report surveys the current state of embedded Rust support on three platforms with deliberately different levels of vendor investment: the Espressif ESP32-C6 (a RISC-V ESP32 variant), the Raspberry Pi RP2350 as found on the Pico 2 W, and the NXP LPC845, the microcontroller currently used in the Control Lab. The same six demonstration programs (LED, switch, UART, ADC, DAC and PWM) are implemented on each platform as far as the hardware allows, and each is used as evidence for a qualitative assessment of crate maturity, documentation, peripheral coverage, toolchain effort and debugging support.

Contents

Abstract	I
List of Figures	III
List of Tables	III
List of Program Code	IV
Abbreviations	V
1 Introduction	1
2 Essentials	1
2.1 Why Rust for Embedded Systems	1
2.2 The Embedded Rust Crate Landscape	2
2.3 Microcontrollers Under Test	3
3 Methodology	5
3.1 Common Program Set	5
3.2 Evaluation Criteria	5
3.3 Development and Toolchain Setup	6
3.4 Build and Flash Verification Workflow	6
4 Implementation and Results	8
4.1 ESP32-C6 (RISC-V)	8
4.2 Raspberry Pi RP2350	16
4.3 NXP LPC845	26
5 Comparative Discussion	36
5.1 Comparison Table	36
5.2 Binary Size	36
5.3 Lessons Learned	38
5.4 Limitations of This Survey	38
6 Conclusion	39
References	39
AI Tool Usage	41
A Repository Layout and Build Instructions	42
B Pin Assignments Used in This Survey	43

List of Figures

2.1	Layering of a typical embedded Rust project	3
3.1	Two-stage verification workflow	6
4.1	RTT/defmt output from the ESP32-C6 ADC demo: the reading tracks the potentiometer, from 4095 at one end down to about 2170 near the midpoint.	13
4.2	RP2350: picotool loading and verifying the ELF over the PICOBOOT interface while the board is held in BOOTSEL	18
4.3	LPC845: echo over the VCOM port exposed by the board's own LPC11U35 debug probe	30

List of Tables

2.1	The three boards used in this survey.	2
3.1	Flashing and live-output options per platform.	7
5.1	Qualitative comparison of Rust support across the three platforms. . .	36
5.2	Per-demo coverage. H = implemented through the HAL, R = implemented against raw PAC registers, – = not possible (no such peripheral on the chip). The denominator counts only demos the hardware supports, so it is 5 for the ESP32-C6 and RP2350 and 6 for the LPC845	36
5.3	Flash and static RAM usage per demo, in bytes.	37
B.1	Pin assignments per demo and platform. “ext.” marks a signal that required external wiring; all other signals are on-board.	43

List of Program Code

3.1	Stage-1 build verification, run from inside each demo project directory.	7
4.1	ESP32-C6 toolchain installation.	8
4.2	ESP32-C6 <code>Cargo.toml</code> dependencies	9
4.3	ESP32-C6 <code>.cargo/config.toml</code>	9
4.4	ESP32-C6: blinking an LED on <code>GPIO7</code>	10
4.5	ESP32-C6: reading the BOOT button on <code>GPIO9</code>	10
4.6	ESP32-C6: asynchronous UART echo on <code>UART1</code>	11
4.7	ESP32-C6: one-shot ADC1 read on <code>GPIO2</code>	12
4.8	ESP32-C6: LEDC-based PWM fade on <code>GPIO3</code>	14
4.9	RP2350 toolchain installation.	16
4.10	RP2350 <code>Cargo.toml</code> dependencies	16
4.11	RP2350 <code>.cargo/config.toml</code>	17
4.12	RP2350 <code>memory.x</code> (excerpt).	17
4.13	RP2350 <code>build.rs</code> : copies <code>memory.x</code> where the linker can find it and adds the RP235x-specific link arguments.	17
4.14	RP2350: blinking the on-board LED through the CYW43439 Wi-Fi chip.	18
4.15	RP2350: interrupt-driven (async) button input.	20
4.16	RP2350: DMA-backed asynchronous UART echo.	21
4.17	RP2350: asynchronous ADC sampling on three channels plus the internal temperature sensor.	22
4.18	RP2350: two PWM slices, one driven by raw compare values and one through the <code>embedded-hal SetDutyCycle</code> trait.	24
4.19	LPC845 toolchain installation.	26
4.20	LPC845 <code>Cargo.toml</code>	26
4.21	LPC845 <code>.cargo/config.toml</code>	26
4.22	LPC845 <code>memory.x</code> : 64 KB of flash from address 0, 16 KB of SRAM from 0x10000000.	27
4.23	LPC845: blinking the on-board red LED on <code>PI01_2</code>	27
4.24	LPC845: reading the User button on <code>PI00_4</code> and mirroring it onto the LED.	28
4.25	LPC845: USART0 echo routed to the on-board VCOM pins through the Switch Matrix.	29
4.26	LPC845: blocking ADC read of the on-board potentiometer on <code>PI00_7</code> .	31
4.27	LPC845: 10-bit DAC0 triangle wave on <code>PI00_17</code> , written directly against the PAC.	32
4.28	LPC845: SCT0-based PWM on <code>PI01_1</code> (the blue LED), written directly against the PAC.	34
A.1	Repository layout.	42

Abbreviations

Abbreviation	Description
ADC	Analog-to-Digital Converter
API	Application Programming Interface
BSP	Board Support Package
CMSIS-DAP	Cortex Microcontroller Software Interface Standard - Debug Access Port
DAC	Digital-to-Analog Converter
DMA	Direct Memory Access
GPIO	General-Purpose Input/Output
HAL	Hardware Abstraction Layer
IOCON	I/O Configuration block (LPC8xx)
JTAG	Joint Test Action Group (debug interface)
LEDC	LED Control peripheral (Espressif PWM unit)
MCU	Microcontroller Unit
PAC	Peripheral Access Crate
PIO	Programmable I/O (RP2350)
PWM	Pulse-Width Modulation
RTT	Real-Time Transfer (debug logging channel)
SAR	Successive Approximation Register (ADC type)
SCT	State Configurable Timer (LPC8xx)
SoC	System on Chip
SWD	Serial Wire Debug
SWM	Switch Matrix (LPC8xx pin routing block)
SYSCON	System Configuration block (LPC8xx)
UART	Universal Asynchronous Receiver/Transmitter
USART	Universal Synchronous/Asynchronous Receiver/Transmitter
VCOM	Virtual Communication Port
PIO	Programmable I/O or a port I/O pin

1 Introduction

Rust is newly part of the curriculum at the Hochschule, and this project serves two purposes. The first is to establish how far the language’s ecosystem has actually matured for embedded work, since that determines whether it is usable for teaching at all. The second is to reach the layers below the abstractions: where no ready driver exists, the peripheral has to be configured register by register, which exposes both what a hardware abstraction layer normally hides and what the language offers once that safety net is gone. One of the three boards surveyed is the microcontroller currently used in the Control Lab, so the results bear directly on whether Rust is a realistic option there.

Rust has emerged over the past several years as a systems programming language that combines the low-level control traditionally associated with C and C++ with strong compile-time guarantees around memory and thread safety. The practical usefulness of Rust for embedded work is only partly determined by the language itself. It also depends heavily on the maturity of the vendor- and community-maintained crates that expose a given microcontroller’s peripherals, on the quality of the accompanying documentation, and on the ergonomics of the toolchain used to build, flash and debug firmware.

This project surveys the current state of embedded Rust support across three microcontroller platforms: the RISC-V variants of Espressif’s ESP32 family, the Raspberry Pi RP2350, and the NXP LPC845.

To make the comparison concrete, the same set of six representative peripheral demos is implemented on each platform: a digital output (LED), a digital input (switch), asynchronous serial communication (UART), analog-to-digital conversion (ADC), digital-to-analog conversion (DAC) and pulse-width modulation (PWM). Each demo is documented, and the experience of implementing it is used to assess the maturity of the respective Rust ecosystem.

The survey also covers how the three platforms differ in how a demo is built, flashed and verified on real hardware.

2 Essentials

2.1 Why Rust for Embedded Systems

Rust’s ownership and borrow-checking rules enforce at compile time that memory is either uniquely owned and mutable or shared and immutable. That rules out dangling pointers, use-after-free and data races between an interrupt handler and the main loop, with no garbage collector and no runtime. Combined with zero-cost abstractions [1],

traits and generics that compile down to something comparable to hand-written C, this gives memory safety without the performance or predictability cost that would rule it out for microcontrollers.

Embedded Rust programs typically run in a `#![no_std]` environment: instead of the full standard library they rely on the `core` and optionally `alloc` crates, and they interact with hardware through peripheral registers rather than through an operating system. This is analogous to bare-metal C development, but with Rust's type system used to encode hardware invariants. A GPIO pin's mode (input, output, alternate function) is represented as a distinct type, so that misusing it becomes a compile error rather than a runtime bug. This pattern is usually called *typestate*, and all three HALs examined in this report use some form of it.

Table 2.1 – The three boards used in this survey.

Board (as provided)	MCU	Architecture used	Remarks
ESP32-C6-DevKit-NX	ESP32-C6	RISC-V (RV32IMAC)	On-board LED is an addressable WS2812, not a plain GPIO
Raspberry Pi Pico 2 W	RP2350	Arm Cortex-M33	On-board LED lives behind the Wi-Fi chip, not on a GPIO at all; no on-board debug probe
LPC845-BRK Rev A	LPC845	Arm Cortex-M0+	No project template and no official HAL, though a community HAL covers most peripherals

2.2 The Embedded Rust Crate Landscape

Embedded Rust projects are typically layered into a small number of crate categories, illustrated in Figure 2.1:

- **Peripheral Access Crate (PAC):** a thin, usually auto-generated (e.g. via `svd2rust`) wrapper around a chip's memory-mapped registers, giving type-safe but very low-level access to every register bit.
- **Hardware Abstraction Layer (HAL) crate:** built on top of the PAC, it implements the vendor-agnostic traits defined by the `embedded-hal` crate family (digital I/O, ADC, SPI, I2C, PWM, ...), so that driver crates written against `embedded-hal` can run on many different chips.
- **Board Support Package (BSP) crate:** an optional further layer that maps HAL functionality onto the pin layout and peripherals of one specific development board.

-
- **Tooling:** `probe-rs` for flashing and on-chip debugging over SWD/JTAG, `picotool` for flashing an RP2350 over its USB bootloader interface, and `defmt` for efficient, embedded-friendly logging.

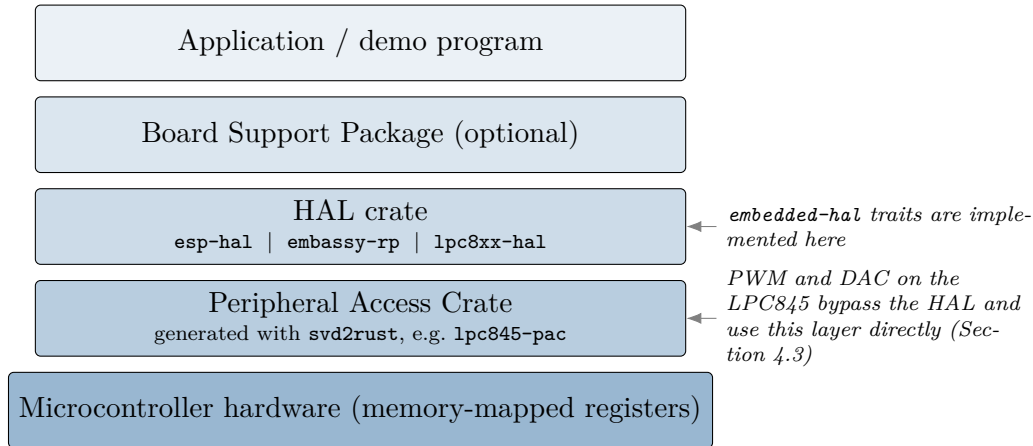


Figure 2.1 – Layering of a typical embedded Rust project

2.3 Microcontrollers Under Test

2.3.1 ESP32 (RISC-V variant)

The ESP32-C6 uses a standard RISC-V core, which means it can be targeted with the upstream, unmodified Rust compiler and the standard `riscv32imc/riscv32imac` targets. Espressif maintains an official `esp-rs` organisation providing a PAC, a HAL (`esp-hal`) and Wi-Fi/Bluetooth crates, along with active documentation. Because of this official backing the RISC-V ESP32 variants are frequently cited as one of the best-supported microcontroller families for Rust; this is examined concretely in this survey.

The board provided was an ESP32-C6-DevKit-NX, an ESP32-C6-DevKitC-1 form-factor board. The ESP32-C6 is a single-core 32-bit RISC-V SoC clocked at up to 160 MHz with 512 KB of on-chip SRAM, integrated Wi-Fi 6, Bluetooth 5 (LE) and 802.15.4 (Zigbee/Thread) radios, a small always-on low-power RISC-V co-processor ([10]), and a built-in USB-Serial/JTAG bridge that allows flashing and `probe-rs` debugging over a single USB cable without an external probe. The peripherals relevant here are 31 GPIO pins, two UART controllers, one seven-channel 12-bit SAR ADC (ADC1), an SPI and an I2C controller, and a six-channel LED PWM controller (LEDC).

The DevKit’s on-board user LED is an addressable WS2812 (“NeoPixel”) wired to GPIO8, also a strapping pin sampled during boot. An external LED on a plain, non-strapping pin was therefore used for the LED, switch and PWM demos, keeping

this survey’s LED demo comparable across all three platforms. The switch demo uses a strapping pin, GPIO9, but only because that is the on-board BOOT button (Section 4.1.3).

2.3.2 Raspberry Pi RP2350

The RP2350 is supported by the `embassy` async embedded framework.

The board used was a Raspberry Pi Pico 2 W (RP2350A package with an on-board Infineon CYW43439 Wi-Fi/Bluetooth radio): dual-core, clocked at up to 150 MHz, 520 KB of on-chip SRAM, 4 MB of external QSPI flash, 26 multi-purpose GPIO pins, 2 UART, 2 SPI and 2 I2C controllers, and 24 PWM channels (see RP2350 datasheet [16]). All demos target the chip’s Arm Cortex-M33 cores (`thumbv8m.main-none-eabi`), which is where `embassy-rp`’s peripheral driver support currently concentrates.

Unlike the other two boards, the Pico 2 W carries no debug probe on the PCB and the RP2350 has no USB-attached debug bridge of its own. The chip enumerates over USB only in BOOTSEL mode, as a mass-storage device plus a PICOBOT interface. `probe-rs` consequently reports no connected probe on this board, and flashing has to go through `picotool` or a UF2 file copy unless an external SWD probe (for example a second Pico running `debugprobe`) is added.

2.3.3 NXP LPC845

The LPC845 is an Arm Cortex-M0+ microcontroller from NXP’s LPC800 series, clocked at up to 30 MHz with 64 KB of flash and 16 KB of SRAM, and it is the microcontroller currently used in the Control Lab. The chip exposes 42 GPIO pins across two ports, five USART instances, two SPI and four I2C controllers, one 12-bit SAR ADC, two 10-bit DACs and a State Configurable Timer (SCT).

The board used was the LPC845-BRK Rev A (NXP part OM40001, documented in UM11181 [14]), NXP’s low-cost breakout board for the LPC845. Like the ESP32-C6-DevKit-NX, it integrates a CMSIS-DAP compatible debug probe. The same probe additionally provides a VCOM (virtual serial port) bridge.

The two Rust crates targeting this chip are `lpc845-pac` and `lpc-rs/lpc8xx-hal`. The `lpc845-pac` [6], referenced in the project brief, is a peripheral-access crate auto-generated with `svd2rust` from NXP’s SVD file. It compiles cleanly and exposes every register defined in the SVD, but, being auto-generated, provides no ergonomic wrapper whatsoever: every peripheral is used through raw read-modify-write access on typed register structs, exactly the C-style register twiddling that Rust’s ownership model is normally used to abstract away.

The `lpc-rs/lpc8xx-hal` [11] explicitly supports both the LPC82x and the LPC845

and implements the `embedded-hal` traits for GPIO, USART, I2C, SPI and ADC. It has neither a PWM/SCT driver nor a DAC driver, however, so both of this survey’s PWM and DAC demos were written directly against `lpc845-pac`’s raw registers (Section 4.3). `lpc8xx-hal` also still targets the older `embedded-hal` 0.2 trait generation rather than the `embedded-hal` 1.0/async generation that `esp-hal` and `embassy-rp` already expose.

3 Methodology

3.1 Common Program Set

To keep the comparison between platforms fair, the same six demo programs are implemented, as far as each platform’s hardware allows, on every microcontroller:

- **LED:** digital output
- **Switch:** digital input
- **UART:** asynchronous serial communication
- **ADC:** analog-to-digital conversion
- **DAC:** digital-to-analog conversion
- **PWM:** pulse-width modulation

3.2 Evaluation Criteria

Each platform is assessed qualitatively against the following criteria, summarised in the comparison table in Section 5.1:

- **Crate maturity:** whether an official or actively maintained PAC/HAL exists, and how recently it was updated.
- **Documentation:** availability and quality of guides, API documentation and working examples.
- **Peripheral coverage:** whether all six demo peripherals are actually exposed by the HAL, and how completely.
- **Toolchain / flashing:** effort required to install the target, build and flash a binary onto the board.
- **Debugging:** availability and quality of tooling such as `probe-rs` and `defmt`-based logging.

3.3 Development and Toolchain Setup

For each platform the general workflow follows the same pattern: install the appropriate Rust target via `rustup`, scaffold a project (by hand, or via `esp-generate` for the ESP32-C6), add the relevant PAC/HAL crates as dependencies, and flash the resulting binary to the board using the platform’s preferred flashing tool. The concrete commands, crate versions and any deviations from this pattern are documented per platform in Section 4.

The host machine used for all three platforms ran Windows with toolchains managed by `rustup`. Three bare-metal targets were installed: `riscv32imac-unknown-none-elf` for the ESP32-C6, `thumbv8m.main-none-eabihf` for the RP2350 and `thumbv6m-none-eabi` for the LPC845. No compiler fork was required for any of them, since the RISC-V ESP32 variants are supported directly by upstream `rustc`. Espressif’s `esp-generate` scaffolding tool was used to bootstrap the ESP32 project from a template; no equivalent tool exists for the other two platforms, so both of those projects were assembled by hand. `esp-generate` also emits a `rust-toolchain.toml` pinning the channel, which the two hand-assembled projects do not have. It was removed here, since the project builds on the pre-installed toolchain and dropping the pin makes the code more portable.

Each board lives in its own directory with its own `Cargo.toml`, `.cargo/config.toml` and (where required) `memory.x`; the three projects are deliberately *not* combined into a single Cargo workspace, because each needs a different default build target and a different linker configuration. The resulting repository layout is given in Appendix A.

3.4 Build and Flash Verification Workflow

Every demo was verified in two stages, as far as the available equipment allowed: a hardware-independent build check that catches most mistakes without the board connected, and a flash-and-observe step performed on the physical board. Figure 3.1 shows both stages and where the three platforms diverge.

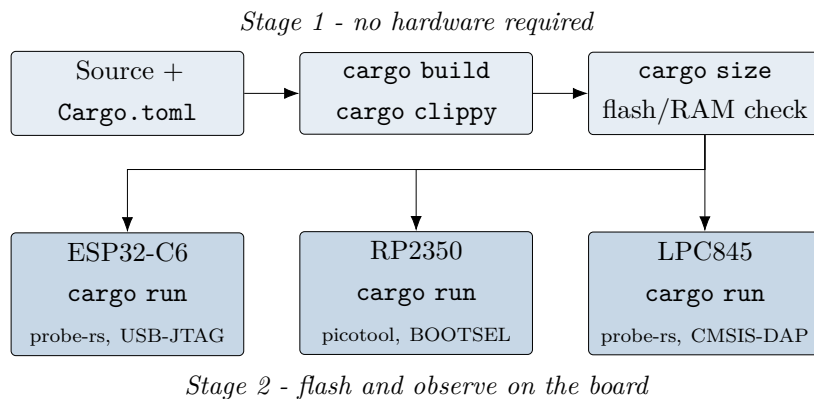


Figure 3.1 – Two-stage verification workflow

3.4.1 Stage 1 - Hardware-Independent Build Verification

This stage catches the majority of mistakes (type errors, missing features, wrong target, API misuse) without a board connected at all.

```
1 cargo build --release          # uses the target from .cargo/config.toml
2 cargo size --release -- -A     # inspect flash/RAM usage per section
3 cargo objdump --release -- -d --no-show-raw-insn # optional: disassembly
4 cargo clippy --release --all-targets           # lint pass, catches HAL
    ↪ misuse
```

Program Code 3.1 – Stage-1 build verification, run from inside each demo project directory.

The flash and RAM figures produced by `cargo size` are collected in Section 5.2. These commands prove useful at any stage when things look inconsistent and there is a need for verification. A clean `cargo build` against the version pinned in `Cargo.lock` is the cheapest way to confirm that the register and pin names used in Section 4.3 still match the installed crate before ever touching the physical board.

3.4.2 Stage 2 - Flashing and Observing on Real Hardware

The ESP32-C6 carries a USB-Serial/JTAG peripheral and can be debugged/flushed without a probe using `probe-rs` ([2]), providing logging capabilities through RTT channel, whereas the Pico 2 W has to be put into BOOTSEL mode by hand before every flash.

Table 3.1 – Flashing and live-output options per platform.

Platform	Flash/run command	Live output capture
ESP32-C6- DevKit-NX	<code>cargo run --release</code> <code>--bin <name></code> (runner: <code>probe-rs</code>)	RTT/defmt streamed by the same command over the built-in USB-Serial/JTAG port
RP2350 (Pico 2 W)	<code>cargo run --release</code> <code>--bin <name></code> (runner: <code>picotool</code>), board held in BOOTSEL	No RTT channel: <code>picotool</code> is flash-only. Verification is limited to the UART demo’s own serial link and to visible LED behaviour
LPC845-BRK Rev A	<code>cargo run --release</code> <code>--bin <name></code> (runner: <code>probe-rs</code>)	No defmt integration in <code>lpc8xx-hal</code> ; output was observed over the board’s own VCOM port (USART0 on PI00_24/PI00_25)

Dependencies needed to flash and observe the boards from the development machine:

- `rustup` with the relevant targets: `riscv32imac-unknown-none-elf`,
`thumbv8m.main-none-eabihf`, `thumbv6m-none-eabi`.

-
- `probe-rs-tools` (`cargo install probe-rs-tools --locked`), which supplies both `cargo-embed` and the `probe-rs` CLI used for RTT capture. Used for the ESP32-C6 and the LPC845-BRK.
 - `picotool`, used only for the RP2350 while it is in BOOTSEL mode. A prebuilt binary is available from the `raspberrypi/pico-sdk-tools` releases; building it from source additionally requires CMake and the `pico-sdk`.
 - A USB-to-serial adapter for the ESP32-C6 and RP2350 UART demos. The LPC845-BRK does not need one, because its on-board probe already exposes a VCOM port.
 - `picocom`, `minicom` or PuTTY as a terminal for the UART demos.

4 Implementation and Results

This section documents, for each of the three platforms, the toolchain setup and the implementation of the six demo peripherals. The pin assignments used across all three boards are collected in Appendix B.

4.1 ESP32-C6 (RISC-V)

4.1.1 Toolchain and Crate Setup

The `esp-hal` crate [8], part of the official `esp-rs` organisation, is the expected starting point, together with `probe-rs` for flashing over USB.

Espressif's `esp-generate` was used to scaffold the project (`esp-generate --chip esp32c6 esp32-c6-dev-kit-nx`), which produces a ready-to-build `no_std` binary crate with the correct target (`riscv32imac-unknown-none-elf`), linker scripts and a minimal `esp_hal::init()` entry point. `esp-hal` was pinned to the 1.x stable line. The `unstable` feature is a checkbox in `esp-generate`'s scaffolding UI rather than a default. Opting in is not really optional here, though. The `analog` and `ledc` modules do not exist without it, so the ADC and PWM demos would be unwritable, and `esp-rtos` declares `esp-hal`'s `requires-unstable` marker feature, which exists purely to produce a clear error when the application has not enabled unstable itself. Only the LED and switch demos could be written on the stable surface alone, and only by replacing the blocking `Delay`, itself gated, with a busy-wait on `esp_hal::time::Instant`. Flashing uses `probe-rs` over the DevKit's built-in USB-Serial/JTAG interface; no external debug probe is needed, and the same port carries RTT/`defmt` log output.

```

1 rustup target add riscv32imac-unknown-none-elf
2 cargo install esp-generate --locked    # project scaffolding tool
3 cargo install probe-rs-tools --locked  # flashing + RTT/defmt over USB-Serial/
    ↪ JTAG
4 cargo install cargo-binutils --locked # optional: cargo size / cargo objdump
5 rustup component add llvm-tools-preview
6 esp-generate --chip esp32c6 esp32-c6-dev-kit-nx

```

Program Code 4.1 – ESP32-C6 toolchain installation.

```

1 [dependencies]
2 esp-hal = { version = "~1.1.0", features = ["defmt", "esp32c6", "unstable"] }
3 defmt   = "1.0.1"
4 esp-bootloader-esp-idf = { version = "0.5.0", features = ["defmt", "esp32c6"] }
5 critical-section = "1.2.0"
6 panic-rtt-target = { version = "0.2.0", features = ["defmt"] }
7 rtt-target       = { version = "0.6.2", features = ["defmt"] }
8 embassy-executor = "0.10.0"
9 embedded-io-async = "0.7.0"
10 esp-rtos = { version = "0.3.0", features = ["embassy", "esp32c6"] }
11 nb = "1.1.0"

```

Program Code 4.2 – ESP32-C6 Cargo.toml dependencies

```

1 [target.riscv32imac-unknown-none-elf]
2 runner = "probe-rs run --chip=esp32c6 --preverify --always-print-stacktrace --no-
    ↪ location"
3
4 [env]
5 DEFMT_LOG="info"
6
7 [build]
8 rustflags = [
9     "-C", "force-frame-pointers",
10 ]
11
12 target = "riscv32imac-unknown-none-elf"

```

Program Code 4.3 – ESP32-C6 .cargo/config.toml.

4.1.2 LED (GPIO Output)

Drive a single GPIO pin high and low to blink an external LED.

```

1  #![no_std]
2  #![no_main]
3
4  use panic_rtt_target as _;
5  use esp_hal::{delay::Delay, gpio::{Level, Output, OutputConfig}, main};
6
7  esp_bootloader_esp_idf::esp_app_desc!();
8
9  #[main]
10 fn main() -> ! {
11     rtt_target::rtt_init_defmt!();
12     let peripherals = esp_hal::init(esp_hal::Config::default());
13     let mut led = Output::new(peripherals.GPIO7, Level::Low, OutputConfig::
        ↪ default());
14     let delay = Delay::new();
15
16     loop {
17         led.toggle();
18         delay.delay_millis(500);
19     }
20 }

```

Program Code 4.4 – ESP32-C6: blinking an LED on GPIO7.

An LED wired through a series resistor to GPIO7 blinks at 1 Hz as expected. GPIO7 was used deliberately instead of the board’s on-board LED: the DevKit’s on-board LED is an addressable WS2812 on GPIO8, not a plain digital output, and driving it correctly requires the RMT peripheral and a small WS2812 protocol driver rather than a one-line `Output::new()/toggle()` call.

4.1.3 Switch (GPIO Input)

Read a digital input (push-button) and mirror its state onto the LED.

```

1  // same as LED demo till here
2  use esp_hal::gpio::{Input, InputConfig, Pull};
3
4  #[main]
5  fn main() -> ! {
6     rtt_target::rtt_init_defmt!();
7
8     let peripherals = esp_hal::init(esp_hal::Config::default());
9     let mut led = Output::new(peripherals.GPIO7, Level::Low, OutputConfig::
        ↪ default());

```

```

10     let button = Input::new(peripherals.GPIO9, InputConfig::default().with_pull(
    ↪ Pull::Up));
11     let delay = Delay::new();
12
13     loop {
14         if button.is_low() {
15             led.set_high();
16         } else {
17             led.set_low();
18         }
19         delay.delay_millis(20);
20     }
21 }

```

Program Code 4.5 – ESP32-C6: reading the BOOT button on GPIO9.

GPIO9 is the DevKit’s on-board BOOT button (active low, already pulled up on the board), which made this demo possible without any extra wiring. Note that GPIO9 is also a strapping pin; holding it low across a reset puts the chip into the serial bootloader, which is harmless here but worth knowing when the same button is reused in an application.

4.1.4 UART

Configure a UART peripheral for asynchronous serial communication, echoing received bytes back over a terminal.

```

1  #![no_std]
2  #![no_main]
3
4  use embassy_executor::Spawner;
5  use embedded_io_async::{Read, Write};
6  use panic_rtt_target as _;
7  use esp_hal::{timer::timg::TimerGroup, uart::{Config as UartConfig, Uart}};
8  use esp_hal::interrupt::software::SoftwareInterruptControl;
9
10 esp_bootloader_esp_idf::esp_app_desc!();
11
12 #[esp_rtos::main]
13 async fn main(_spawner: Spawner) {
14     rtt_target::rtt_init_defmt!();
15     let peripherals = esp_hal::init(esp_hal::Config::default());
16     let timg0 = TimerGroup::new(peripherals.TIMG0);
17     let software_interrupt = SoftwareInterruptControl::new(peripherals.

```

```

    ↪ SW_INTERRUPT);
18     esp_rtos::start(timg0.timer0, software_interrupt.software_interrupt0);
19
20     let mut uart = Uart::new(peripherals.UART1, UartConfig::default())
21         .unwrap()
22         .with_rx(peripherals.GPIO1)
23         .with_tx(peripherals.GPIO0)
24         .into_async();
25
26     let mut buf = [0u8; 1];
27     loop {
28         Read::read(&mut uart, &mut buf).await.unwrap();
29         Write::write_all(&mut uart, &buf).await.unwrap();
30     }
31 }

```

Program Code 4.6 – ESP32-C6: asynchronous UART echo on UART1.

A second UART controller, UART1, was configured on GPIO0 (TX) and GPIO1 (RX) and wired to a USB-to-serial adapter, keeping it independent of the chip’s built-in USB-Serial/JTAG connection used for RTT logging. Every character typed in a terminal at 115 200 8N1 is echoed back. Getting this running needed one dependency the LED and switch demos did not: `esp-rtos`, which supplies the async executor integration that `esp-hal`’s UART driver needs.

4.1.5 ADC

Sample an analog input voltage and convert it to a digital value.

```

1  #![no_std]
2  #![no_main]
3
4  use panic_rtt_target as _;
5  use esp_hal::{
6      analog::adc::{Adc, AdcConfig, Attenuation},
7      delay::Delay,
8      main,
9  };
10
11 esp_bootloader_esp_idf::esp_app_desc!();
12
13 #[main]
14 fn main() -> ! {
15     rtt_target::rtt_init_defmt!();

```

```

16  let peripherals = esp_hal::init(esp_hal::Config::default());
17  let mut adc_config = AdcConfig::new();
18  let mut pin = adc_config.enable_pin(peripherals.GPIO2, Attenuation::_11dB);
19  let mut adc1 = Adc::new(peripherals.ADC1, adc_config);
20  let delay = Delay::new();
21
22  loop {
23      let value: u16 = nb::block!(adc1.read_oneshot(&mut pin)).unwrap();
24      defmt::info!("adc: {}", value);
25      delay.delay_millis(200);
26  }
27 }

```

Program Code 4.7 – ESP32-C6: one-shot ADC1 read on GPIO2.

A potentiometer wiper on GPIO2 (ADC1 channel 2) produced a raw 12-bit code between 0 and 4095 that tracked the wiper smoothly, logged over RTT rather than a serial console. Figure 4.1 shows a representative capture: the reading holds at 4095 while the potentiometer is turned fully clockwise, then settles near 2170 at roughly its midpoint. The small jitter between readings is typical of a real analog signal, not a fixed value.



```

[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 2169
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 4095
[INFO ] adc: 2170
[INFO ] adc: 2170
[INFO ] adc: 2169
[INFO ] adc: 2171
[INFO ] adc: 2171
[INFO ] adc: 2170
[INFO ] adc: 2169
[INFO ] adc: 2172
[INFO ] adc: 2170
[INFO ] adc: 2169
[INFO ] adc: 2175
[INFO ] adc: 2171
[INFO ] adc: 2169
[INFO ] adc: 2170
[INFO ] adc: 2171
[INFO ] adc: 2170
[INFO ] adc: 2169

```

Figure 4.1 – RTT/defmt output from the ESP32-C6 ADC demo: the reading tracks the potentiometer, from 4095 at one end down to about 2170 near the midpoint.

4.1.6 DAC

Generate an analog output voltage from a digital value.

The RISC-V ESP32 variants have no on-chip DAC: the 8-bit DAC peripheral exists only on the original Xtensa-based ESP32 and the ESP32-S2, so `esp-hal`'s `dac` module is

compiled in for the `esp32` and `esp32s2` targets only, not for `esp32c6`. As a workaround, the LEDC PWM output used for the demo below was routed through a first-order RC low-pass filter ($1\text{ k}\Omega / 10\text{ }\mu\text{F}$, corner frequency $f_c = 1/(2\pi RC) \approx 16\text{ Hz}$, i.e. far below the 1 kHz PWM carrier) on `GPI03`. Sweeping the duty cycle from 0% to 100% should produce a filtered output ramping smoothly between roughly 0 V and the 3.3 V supply, with some drop below the rail expected at full duty.

4.1.7 PWM

Generate a pulse-width-modulated signal, for example to dim an LED.

```
1  #![no_std]
2  #![no_main]
3
4  use panic_rtt_target as _;
5  use esp_hal::{
6      delay::Delay,
7      gpio::DriveMode,
8      ledc::{channel, channel::ChannelIFace, timer, timer::TimerIFace,
9          ↪ LSGlobalClkSource, Ldc, LowSpeed},
10     main,
11     time::Rate,
12 };
13 esp_bootloader_esp_idf::esp_app_desc!();
14
15 #[main]
16 fn main() -> ! {
17     rtt_target::rtt_init_defmt!();
18     let peripherals = esp_hal::init(esp_hal::Config::default());
19     let mut ledc = Ldc::new(peripherals.LEDC);
20     ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);
21
22     let mut lstimer0 = ledc.timer::<LowSpeed>(timer::Number::Timer0);
23     lstimer0
24         .configure(timer::config::Config {
25             duty: timer::config::Duty::Duty14Bit,
26             clock_source: timer::LSClockSource::APBClk,
27             frequency: Rate::from_khz(1),
28         })
29         .unwrap();
30
31     let mut channel0 = ledc.channel(channel::Number::Channel0, peripherals.GPI03
32         ↪ );
```

```

32     channel0
33         .configure(channel::config::Config {
34             timer: &lstimer0,
35             duty_pct: 0,
36             drive_mode: DriveMode::PushPull,
37         })
38         .unwrap();
39
40     let delay = Delay::new();
41     let mut duty: u8 = 0;
42     let mut rising = true;
43     loop {
44         channel0.set_duty(duty).unwrap();
45         if rising {
46             duty += 1;
47             if duty == 100 { rising = false; }
48         } else {
49             duty -= 1;
50             if duty == 0 { rising = true; }
51         }
52         delay.delay_millis(20);
53     }
54 }

```

Program Code 4.8 – ESP32-C6: LEDC-based PWM fade on GPIO3.

The LED on GPIO3 fades smoothly up and down, comparable to the RP2350's breathing-LED result. GPIO3 was chosen rather than GPIO4 because GPIO4 is a strap-ping pin (MTMS) on the ESP32-C6 and is best left alone unless needed. On this chip the LEDC peripheral exposes only the low-speed clock domain, so the `LSGlobalClkSource/LowSpeed` setup is the only option.

4.1.8 Summary and Assessment

Rust support for the RISC-V ESP32 variant was strong. The four peripherals covered by `esp-hal`'s stabilised 1.0 API surface (GPIO, UART, SPI, I2C) plus the `#[main]` macro get a project running with very little boilerplate, and the DevKit's built-in USB-Serial/JTAG bridge meant that flashing and logging worked over a single USB cable with no external probe.

`Esp-generate` offers `unstable` as an opt-in checkbox, but every demo in this survey ends up needing it. ADC and LEDC are the obvious cases, since those modules are absent without the feature, but so are the blocking `Delay` and the timer and software-interrupt handles that `esp-rtos` requires to start the async executor, which means

the stabilised 1.0 surface on its own is not quite enough to write even the LED demo as shown. Also, the async UART demo required pulling in `esp-rtos` as a bridging crate, plus an explicit `start()` call that `embassy-rp` does not need, because it owns its executor integration end to end. Despite these issues, every demo has a working ESP32 equivalent.

Crates used: `esp-hal 1.1`, `esp-rtos`, `embassy-executor`, `esp-bootloader-esp-idf`, `panic-rtt-target`, `rtt-target`, `defmt` and `embedded-io-async`.

4.2 Raspberry Pi RP2350

4.2.1 Toolchain and Crate Setup

All demos, except PWM (uses blocking), are built on the async `embassy-rp` HAL from the `embassy-rs/embassy` repository [7], targeting the chip's Arm Cortex-M33 cores. Using the RP2350's RISC-V cores in Rust currently means dropping down to the lower-level `rp235x-hal` crate instead, which trades the async ergonomics used throughout this section for more direct hardware access.

Flashing goes through `picotool` while the board is held in BOOTSEL [3], entered by holding the BOOTSEL button during a power-up or reset. The running program is what leaves BOOTSEL, so this has to be repeated before every flash. `picotool` was preferred over a UF2 file copy because `cargo run` can invoke it directly. Figure 4.2 shows the resulting output.

```
1 rustup target add thumbv8m.main-none-eabihf
```

Program Code 4.9 – RP2350 toolchain installation.

```
1 [dependencies]
2 embassy-executor = { version = "0.10.0", features = ["defmt", "platform-cortex-m
   ↳ ", "executor-thread"] }
3 embassy-rp      = { version = "0.10.0", features = ["defmt", "rp235xa", "binary-
   ↳ info", "time-driver", "critical-section-impl"] }
4 embassy-time    = { version = "0.5.1", features = ["defmt"] }
5 defmt           = "1.0.1"
6 defmt-rtt       = "1.0.0"
7 panic-probe     = { version = "1.0.0", features = ["print-defmt"] }
8 cortex-m-rt     = "0.7.0"
9 cortex-m        = { version = "0.7.6", features = ["inline-asm"] }
10 cyw43           = { version = "0.7.0", features = ["defmt", "firmware-logs"] }
11 cyw43-pio       = { version = "0.10.0", features = ["defmt"] }
```

```
12 static_cell      = "2"
```

Program Code 4.10 – RP2350 Cargo.toml dependencies

```
1 [target.'cfg(all(target_arch = "arm", target_os = "none"))']
2 runner = "picotool load -u -v -x -t elf"
3
4 [build]
5 target = "thumbv8m.main-none-eabihf"
6
7 [env]
8 DEFMT_LOG = "debug"
```

Program Code 4.11 – RP2350 .cargo/config.toml

The RP2350 also needs a linker script. The `memory.x` used here follows the version in embassy's `examples/rp235x`; the essential part is the flash and RAM layout plus the three extra sections the boot ROM and picotool expect. The extra sections are omitted here for brevity.

```
1 MEMORY {
2
3     FLASH : ORIGIN = 0x10000000, LENGTH = 4096K
4
5     RAM : ORIGIN = 0x20000000, LENGTH = 512K
6
7     SRAM8 : ORIGIN = 0x20080000, LENGTH = 4K
8     SRAM9 : ORIGIN = 0x20081000, LENGTH = 4K
9 }
```

Program Code 4.12 – RP2350 memory.x (excerpt).

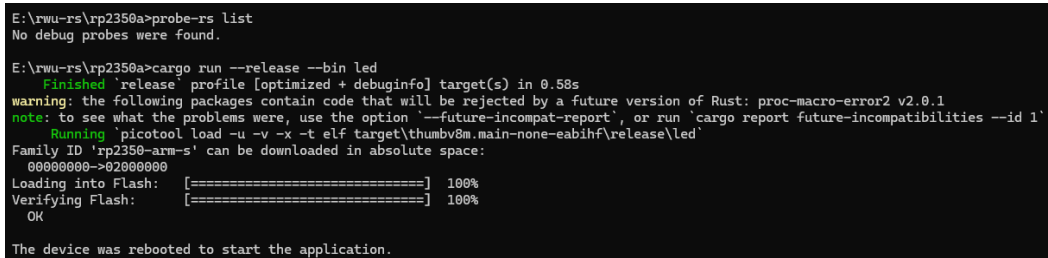
```
1 use std::env;
2 use std::fs::File;
3 use std::io::Write;
4 use std::path::PathBuf;
5
6 fn main() {
7
8     let out = &PathBuf::from(env::var_os("OUT_DIR").unwrap());
9     File::create(out.join("memory.x"))
10         .unwrap()
11         .write_all(include_bytes!("memory.x"))
```

```

12     .unwrap();
13     println!("cargo:rustc-link-search={}", out.display());
14
15     println!("cargo:rerun-if-changed=memory.x");
16
17     println!("cargo:rustc-link-arg-bins=--nmagic");
18     println!("cargo:rustc-link-arg-bins=-Tlink.x");
19     println!("cargo:rustc-link-arg-bins=-Tdefmt.x");
20 }

```

Program Code 4.13 – RP2350 *build.rs*: copies *memory.x* where the linker can find it and adds the RP235x-specific link arguments.



```

E:\rwm-rs\rp2350a>probe-rs list
No debug probes were found.

E:\rwm-rs\rp2350a>cargo run --release --bin led
Finished release profile [optimized + debuginfo] target(s) in 0.58s
warning: the following packages contain code that will be rejected by a future version of Rust: proc-macro-error2 v2.0.1
note: to see what the problems were, use the option '--future-incompat-report', or run 'cargo report future-incompatibilities --id 1'
Running 'picotool load -u -v -x -t elf target\thumbv8m.main-none-eabihf\release\led'
Family ID 'rp2350-arm-s' can be downloaded in absolute space:
00000000->02000000
Loading into Flash: [=====] 100%
Verifying Flash: [=====] 100%
OK
The device was rebooted to start the application.

```

Figure 4.2 – RP2350: *picotool* loading and verifying the ELF over the *PICOBOT* interface while the board is held in *BOOTSEL*

4.2.2 LED (GPIO Output)

On the Pico 2 W, GPIO25 is the SPI chip-select line to the on-board CYW43439 Wi-Fi chip and the user LED is wired to that chip's own GPIO0, so lighting it means bringing up the Wi-Fi chip's SPI link and firmware first [7].

```

1  #![no_std]
2  #![no_main]
3
4  use cyw43::aligned_bytes;
5  use cyw43_pio::{PioSpi, RM2_CLOCK_DIVIDER};
6  use defmt::*;
7  use embassy_executor::Spawner;
8  use embassy_rp::gpio::{Level, Output};
9  use embassy_rp::peripherals::{DMA_CH0, DMA_CH1, PI00};
10 use embassy_rp::pio::{InterruptHandler, Pio};
11 use embassy_rp::{bind_interrupts, dma};
12 use embassy_time::{Duration, Timer};
13 use static_cell::StaticCell;
14 use {defmt_rtt as _, panic_probe as _};
15

```

```

16
17 #[unsafe(link_section = ".bi_entries")]
18 #[used]
19 pub static PICOTOOL_ENTRIES: [embassy_rp::binary_info::EntryAddr; 4] = [
20     embassy_rp::binary_info::rp_program_name!(c"Blinky Example"),
21     embassy_rp::binary_info::rp_program_description!(
22         c"This example tests the RP Pico 2 W's onboard LED, connected to GPIO 0
    ↪ of the cyw43 \
23         (WiFi chip) via PIO 0 over the SPI bus."
24     ),
25     embassy_rp::binary_info::rp_cargo_version!(),
26     embassy_rp::binary_info::rp_program_build_attribute!(),
27 ];
28
29 bind_interrupts!(struct Irqs {
30     PIO0_IRQ_0 => InterruptHandler<PIO0>;
31     DMA_IRQ_0 => dma::InterruptHandler<DMA_CH0>, dma::InterruptHandler<DMA_CH1>;
32 });
33
34 #[embassy_executor::task]
35 async fn cyw43_task(
36     runner: cyw43::Runner<'static, cyw43::SpiBus<Output<'static>, PioSpi<'static,
    ↪ PIO0, 0>>>,
37 ) -> ! {
38     runner.run().await
39 }
40
41 #[embassy_executor::main]
42 async fn main(spawner: Spawner) {
43     let p = embassy_rp::init(Default::default());
44     let fw = aligned_bytes!("../../cyw43-firmware/43439A0.bin");
45     let clm = aligned_bytes!("../../cyw43-firmware/43439A0_clm.bin");
46     let nvram = aligned_bytes!("../../cyw43-firmware/nvram_rp2040.bin");
47
48     let pwr = Output::new(p.PIN_23, Level::Low);
49     let cs = Output::new(p.PIN_25, Level::High);
50     let mut pio = Pio::new(p.PIO0, Irqs);
51     let spi = PioSpi::new(
52         &mut pio.common,
53         pio.sm0,
54         RM2_CLOCK_DIVIDER,
55         pio.irq0,
56         cs,
57         p.PIN_24,
58         p.PIN_29,
59         dma::Channel::new(p.DMA_CH0, Irqs),

```

```

60     );
61
62     static STATE: StaticCell<cyw43::State> = StaticCell::new();
63     let state = STATE.init(cyw43::State::new());
64     let (_net_device, mut control, runner) = cyw43::new(state, pwr, spi, fw,
65     ↪ nvrn).await;
66     spawner.spawn(unwrap!(cyw43_task(runner)));
67     control.init(clm).await;
68     control
69         .set_power_management(cyw43::PowerManagementMode::PowerSave)
70         .await;
71
72     let delay = Duration::from_millis(250);
73     loop {
74         info!("led on!");
75         control.gpio_set(0, true).await;
76         Timer::after(delay).await;
77
78         info!("led off!");
79         control.gpio_set(0, false).await;
80         Timer::after(delay).await;
81     }
82 }

```

Program Code 4.14 – RP2350: blinking the on-board LED through the CYW43439 Wi-Fi chip.

The on-board LED blinks at 2 Hz. It required locating and embedding the three CYW43439 firmware blobs, wiring up a PIO-based SPI peripheral (PIN_23/24/25/29) and completing an async `control.init()` handshake before a single GPIO could be toggled. An external LED on any free GPIO is a one-liner (`Output::new(p.PIN_15, Level::Low)`) and is what the switch demo below uses.

4.2.3 Switch (GPIO Input)

Read a push-button as a digital input and toggle an LED on each press [7].

```

1  #![no_std]
2  #![no_main]
3
4  use embassy_executor::Spawner;
5  use embassy_rp::gpio::{Input, Level, Output, Pull};
6  use embassy_time::Timer;
7  use {defmt_rtt as _, panic_probe as _};
8
9  #[embassy_executor::main]

```

```

10 async fn main(_spawner: Spawner) {
11     let p = embassy_rp::init(Default::default());
12     let mut led = Output::new(p.PIN_15, Level::Low);
13     let mut button = Input::new(p.PIN_16, Pull::Up);
14
15     loop {
16         button.wait_for_falling_edge().await;
17         Timer::after_millis(20).await; // debounce
18         led.toggle();
19     }
20 }

```

Program Code 4.15 – RP2350: interrupt-driven (async) button input.

The demo waits on an edge instead of polling, so no interrupt handler or shared flag is written by hand.

4.2.4 UART

Configure UART0 and echo every received byte back to the sender [7].

```

1  #![no_std]
2  #![no_main]
3
4  use embassy_executor::Spawner;
5  use embassy_rp::bind_interrupts;
6  use embassy_rp::dma;
7  use embassy_rp::peripherals::{DMA_CH0, DMA_CH1, UART0};
8  use embassy_rp::uart::{Config, InterruptHandler, Uart};
9  use {defmt_rtt as _, panic_probe as _};
10
11 bind_interrupts!(struct Irqs {
12     UART0_IRQ => InterruptHandler<UART0>;
13     DMA_IRQ_0 => dma::InterruptHandler<DMA_CH0>, dma::InterruptHandler<DMA_CH1>;
14 });
15
16 #[embassy_executor::main]
17 async fn main(_spawner: Spawner) {
18     let p = embassy_rp::init(Default::default());
19     let mut uart = Uart::new(
20         p.UART0, p.PIN_0, p.PIN_1, Irqs, p.DMA_CH0, p.DMA_CH1, Config::default(),
21         ↪
22     );
23
24     uart.write(b"Hello World!\r\n").await.unwrap();

```

```

24
25     let mut buf = [0u8; 1];
26     loop {
27         uart.read(&mut buf).await.unwrap();
28         uart.write(&buf).await.unwrap();
29     }
30 }

```

Program Code 4.16 – RP2350: DMA-backed asynchronous UART echo.

Tested against a USB-to-serial adapter at the default 115 200 8N1: every character typed in a terminal is echoed back. DMA channels are handed to the driver explicitly and used transparently to implement the `async read()/write()` futures.

4.2.5 ADC

Sample analog inputs and the on-chip temperature sensor [7].

```

1  #![no_std]
2  #![no_main]
3
4  use defmt::*;
5  use embassy_executor::Spawner;
6  use embassy_rp::adc::{Adc, Channel, Config, InterruptHandler};
7  use embassy_rp::bind_interrupts;
8  use embassy_rp::gpio::Pull;
9  use embassy_time::Timer;
10 use {defmt_rtt as _, panic_probe as _};
11
12 bind_interrupts!(struct Irqs {
13     ADC_IRQ_FIFO => InterruptHandler;
14 });
15
16 #[embassy_executor::main]
17 async fn main(_spawner: Spawner) {
18     let p = embassy_rp::init(Default::default());
19     let mut adc = Adc::new(p.ADC, Irqs, Config::default());
20
21     let mut p26 = Channel::new_pin(p.PIN_26, Pull::None);
22     let mut p27 = Channel::new_pin(p.PIN_27, Pull::None);
23     let mut p28 = Channel::new_pin(p.PIN_28, Pull::None);
24     let mut ts = Channel::new_temp_sensor(p.ADC_TEMP_SENSOR);
25
26     loop {
27         let level = adc.read(&mut p26).await.unwrap();

```

```

28     info!("Pin 26 ADC: {}", level);
29     let level = adc.read(&mut p27).await.unwrap();
30     info!("Pin 27 ADC: {}", level);
31     let level = adc.read(&mut p28).await.unwrap();
32     info!("Pin 28 ADC: {}", level);
33     let temp = adc.read(&mut ts).await.unwrap();
34     info!("Temp: {} degrees", convert_to_celsius(temp));
35     Timer::after_secs(1).await;
36 }
37 }
38
39 fn convert_to_celsius(raw_temp: u16) -> f32 {
40
41     let temp = 27.0 - (raw_temp as f32 * 3.3 / 4096.0 - 0.706) / 0.001721;
42     let sign = if temp < 0.0 { -1.0 } else { 1.0 };
43     let rounded_temp_x10: i16 = ((temp * 10.0) + 0.5 * sign) as i16;
44     (rounded_temp_x10 as f32) / 10.0
45 }

```

Program Code 4.17 – RP2350: asynchronous ADC sampling on three channels plus the internal temperature sensor.

The demo samples three external channels plus the internal temperature sensor and logs each raw 12-bit code. Converting the raw code to a voltage is left to the application; the HAL returns only the raw ADC code. Since there is no RTT channel on this board (Section 3.4), the `info!` output above was not observable during the survey unless an external SWD probe was attached.

4.2.6 DAC

Neither the RP2350 nor the RP2040 before it has a dedicated on-chip DAC peripheral, and the ADC-capable pins are input-only. This is purely a hardware limitation, so there is no corresponding official example even in the **embassy** repository. As on the ESP32-C6, a PWM output through an external RC low-pass filter is the standard workaround, and the same 1 k Ω / 10 μ F filter used in Section 4.1.6 would apply unchanged to either PWM pin in Section 4.2.7. It was not built on this board, because the result is an analog voltage and, without an RTT channel or a bench meter, there would have been nothing to observe beyond what the ESP32-C6 version already demonstrates.

4.2.7 PWM

Drive two PWM slices: one that changes its compare value directly and one that uses the `embedded-hal` `SetDutyCycle` trait [7]. Note one pin change from the reference

implementation: PIN_25 is the CYW43439 chip-select line on the W variant of this board, so GPIO15 (PWM slice 7, channel B) was used instead.

```
1  #![no_std]
2  #![no_main]
3
4  use defmt::*;
5  use embassy_executor::Spawner;
6  use embassy_rp::Peri;
7  use embassy_rp::peripherals::{PIN_4, PIN_15, PWM_SLICE2, PWM_SLICE7};
8  use embassy_rp::pwm::{Config, Pwm, SetDutyCycle};
9  use embassy_time::Timer;
10 use {defmt_rtt as _, panic_probe as _};
11
12 #[embassy_executor::main]
13 async fn main(spawner: Spawner) {
14     let p = embassy_rp::init(Default::default());
15     spawner.spawn(pwm_set_config(p.PWM_SLICE7, p.PIN_15).unwrap());
16     spawner.spawn(pwm_set_dutycycle(p.PWM_SLICE2, p.PIN_4).unwrap());
17 }
18
19 #[embassy_executor::task]
20 async fn pwm_set_config(slice7: Peri<'static, PWM_SLICE7>, pin15: Peri<'static,
    ↪ PIN_15>) {
21     let mut c = Config::default();
22     c.top = 32_768;
23     c.compare_b = 8;
24     let mut pwm = Pwm::new_output_b(slice7, pin15, c.clone());
25
26     loop {
27         info!("current LED duty cycle: {}/32768", c.compare_b);
28         Timer::after_secs(1).await;
29         c.compare_b = c.compare_b.rotate_left(4);
30         pwm.set_config(&c);
31     }
32 }
33
34 #[embassy_executor::task]
35 async fn pwm_set_dutycycle(slice2: Peri<'static, PWM_SLICE2>, pin4: Peri<'static,
    ↪ PIN_4>) {
36     let desired_freq_hz = 25_000;
37     let clock_freq_hz = embassy_rp::clocks::clk_sys_freq();
38     let divider = 16u8;
39     let period = (clock_freq_hz / (desired_freq_hz * divider as u32)) as u16 -
    ↪ 1;
```

```

40
41     let mut c = Config::default();
42     c.top = period;
43     c.divider = divider.into();
44
45     let mut pwm = Pwm::new_output_a(slice2, pin4, c.clone());
46
47     loop {
48         pwm.set_duty_cycle_fully_on().unwrap();
49         Timer::after_secs(1).await;
50
51         pwm.set_duty_cycle_percent(66).unwrap();
52         Timer::after_secs(1).await;
53
54         pwm.set_duty_cycle(c.top / 4).unwrap();
55         Timer::after_secs(1).await;
56
57         pwm.set_duty_cycle_fully_off().unwrap();
58         Timer::after_secs(1).await;
59     }
60 }

```

Program Code 4.18 – RP2350: two PWM slices, one driven by raw compare values and one through the `embedded-hal SetDutyCycle` trait.

The first task steps the compare value of slice 7 once per second by rotating its bit pattern, producing a visibly changing brightness on GPIO15; the second task steps slice 2 through 100 %, 66 %, 25 % and 0 % using the `embedded-hal SetDutyCycle` trait, which is the more portable of the two styles. Replacing the rotation with a linear ramp of `compare_b` from 0 to `top` gives the same smooth breathing effect as the ESP32-C6 demo. Unlike the async LED, switch, UART and ADC APIs, updating a PWM duty cycle is a plain blocking register write, which is appropriate here because there is nothing to wait on.

4.2.8 Summary and Assessment

The RP2350’s async API is provided by the same project that provides the executor, so no bridging crate and no explicit start call are needed, and none of the peripherals used here sit behind a feature flag. Five of the six demos have a directly corresponding, actively maintained official example in `examples/rp235x` of the `embassy-rs/embassy` repository; the sixth (DAC) is simply not applicable, because the chip has no DAC peripheral. The async APIs made the switch and UART demos shorter and easier to reason about than an interrupt-and-flag-based C equivalent.

Since the Pico 2 W has no on-board debug probe, every flash requires physically entering BOOTSEL mode, and the `defmt-rtt` logging that the examples are written around cannot be observed at all without adding an external SWD probe. The practical effect on this survey was that the ADC and PWM demos could only be verified as building and running, since their results are numeric or analog and there was no channel on which to observe them. Adding a second Pico flashed with `debugprobe`, or using a Pico 2 (non-W) plus any CMSIS-DAP probe, would remove this limitation entirely.

4.3 NXP LPC845

4.3.1 Toolchain and Crate Setup

As noted in Section 2.3.3, no official NXP-maintained Rust HAL exists for this chip. The community crate `lpc8xx-hal` [11] covers four of the six demo peripherals, and the remaining two (DAC and PWM) are implemented directly against the auto-generated `lpc845-pac` register API [12].

```
1 rustup target add thumbv6m-none-eabi
2 cargo install probe-rs-tools --locked # supplies cargo-embed and the probe-rs
    ↪ CLI
```

Program Code 4.19 – LPC845 toolchain installation.

```
1 [dependencies]
2 lpc8xx-hal = { version = "0.10", features = ["845-rt"] }
3 cortex-m = "0.7"
4 cortex-m-rt = "0.7"
5 panic-halt = "0.2"
6 nb = "1"
7 embedded-hal = "0.2"
```

Program Code 4.20 – LPC845 Cargo.toml.

```
1 [target.thumbv6m-none-eabi]
2 runner = "probe-rs run --chip LPC845M301JBD48"
3 rustflags = ["-C", "link-arg=-Tlink.x"]
4
5 [build]
6 target = "thumbv6m-none-eabi"
```

Program Code 4.21 – LPC845 .cargo/config.toml.

```

1 MEMORY
2 {
3     FLASH : ORIGIN = 0x00000000, LENGTH = 64K
4     RAM    : ORIGIN = 0x10000000, LENGTH = 16K
5 }

```

Program Code 4.22 – LPC845 memory.x: 64 KB of flash from address 0, 16 KB of SRAM from 0x10000000.

One caveat is that, unlike the other two platforms, no `defmt` integration ships with `lpc8xx-hal`, so logging falls back to the board’s VCOM UART.

4.3.2 LED (GPIO Output)

Drive a single GPIO pin to blink the red segment of the on-board RGB LED.

```

1  #![no_std]
2  #![no_main]
3  use panic_halt as _;
4  use cortex_m_rt::entry;
5  use lpc8xx_hal::{cortex_m::peripheral::Peripherals as CorePeripherals, prelude
   ↪  ::*, delay::Delay, gpio::Level, Peripherals};
6
7  #[entry]
8  fn main() -> ! {
9      let cp = CorePeripherals::take().unwrap();
10     let p = Peripherals::take().unwrap();
11
12     let mut syscon = p.SYSCON.split();
13     let gpio = p.GPIO.enable(&mut syscon.handle);
14     let pins = p.pins;
15
16     // active low: High is LED off
17     let mut led = pins.pio1_2.into_output_pin(gpio.tokens.pio1_2, Level::High);
18     let mut delay = Delay::new(cp.SYST);
19
20     loop {
21         led.toggle();
22         delay.delay_ms(500u32);
23     }
24 }

```

Program Code 4.23 – LPC845: blinking the on-board red LED on PIO1_2.

PIO1_2 drives the red segment of the LPC845-BRK's RGB LED. Structurally this demo needs more setup than either the ESP32 or the RP2350 version: `SYSCON.split()` and `GPIO.enable()` are two explicit steps (clock-gating the GPIO block and handing back a typed handle) that `esp_hal::init()` and `embassy_rp::init()` collapse into one call, and the pin is moved through a typestate transition (`into_output_pin`) rather than constructed directly from a `Level`.

4.3.3 Switch (GPIO Input)

Read a digital input (push-button) with a simple software debounce.

```
1 // same as LED demo till here
2
3 #[entry]
4 fn main() -> ! {
5     let cp = CorePeripherals::take().unwrap();
6     let p = Peripherals::take().unwrap();
7
8     let mut syscon = p.SYSCON.split();
9     let gpio = p.GPIO.enable(&mut syscon.handle);
10    let pins = p.pins;
11
12    // LED is active low, button is active low with an on-board pull-up
13    let mut led = pins.pio1_2.into_output_pin(gpio.tokens.pio1_2, Level::High);
14    let button = pins.pio0_4.into_input_pin(gpio.tokens.pio0_4);
15    let mut delay = Delay::new(cp.SYST);
16
17    loop {
18        if button.is_low() {
19            led.set_low(); // pressed: LED on
20        } else {
21            led.set_high(); // released: LED off
22        }
23        delay.delay_ms(20u32); // simple software debounce
24    }
25 }
```

Program Code 4.24 – LPC845: reading the User button on PIO0_4 and mirroring it onto the LED.

PIO0_4 is the LPC845-BRK's User button (K3), active low with a 10 k Ω pull-up on the board [14], so just as with the ESP32's BOOT button, no extra wiring or pull configuration is needed. `lpc8xx-hal`'s GPIO input implements `embedded-hal`'s blocking `InputPin` (`is_low()`/`is_high()`) and offers no interrupt-driven `wait_for_`-

`edge()` equivalent at this version, so the same fixed-interval polling loop used for the ESP32 switch demo was used here too.

4.3.4 UART

Configure a USART peripheral for serial communication and echo received bytes back over a terminal.

```
1  #![no_std]
2  #![no_main]
3
4  use panic_halt as _;
5  use cortex_m_rt::entry;
6  use lpc8xx_hal::prelude::*;
7  use lpc8xx_hal::{usart, Peripherals};
8  use nb::block;
9
10 #[entry]
11 fn main() -> ! {
12     let p = Peripherals::take().unwrap();
13
14     let mut syscon = p.SYSCON.split();
15     let mut swm = p.SWM.split();
16     let mut swm_handle = swm.handle.enable(&mut syscon.handle);
17
18     let (u0_rxd, _) = swm.movable_functions.u0_rxd.assign(
19         p.pins.pio0_24.into_swm_pin(), &mut swm_handle,
20     );
21     let (u0_txd, _) = swm.movable_functions.u0_txd.assign(
22         p.pins.pio0_25.into_swm_pin(), &mut swm_handle,
23     );
24
25     let clock_config = usart::Clock::new_with_baudrate(115_200);
26     let mut usart0 = p.USART0.enable_async(
27         &clock_config,
28         &mut syscon.handle,
29         u0_rxd,
30         u0_txd,
31         usart::Settings::default(),
32     );
33
34     loop {
35         let byte = block!(usart0.read()).unwrap();
36         block!(usart0.write(byte)).unwrap();
37     }
```

Program Code 4.25 – LPC845: USART0 echo routed to the on-board VCOM pins through the Switch Matrix.

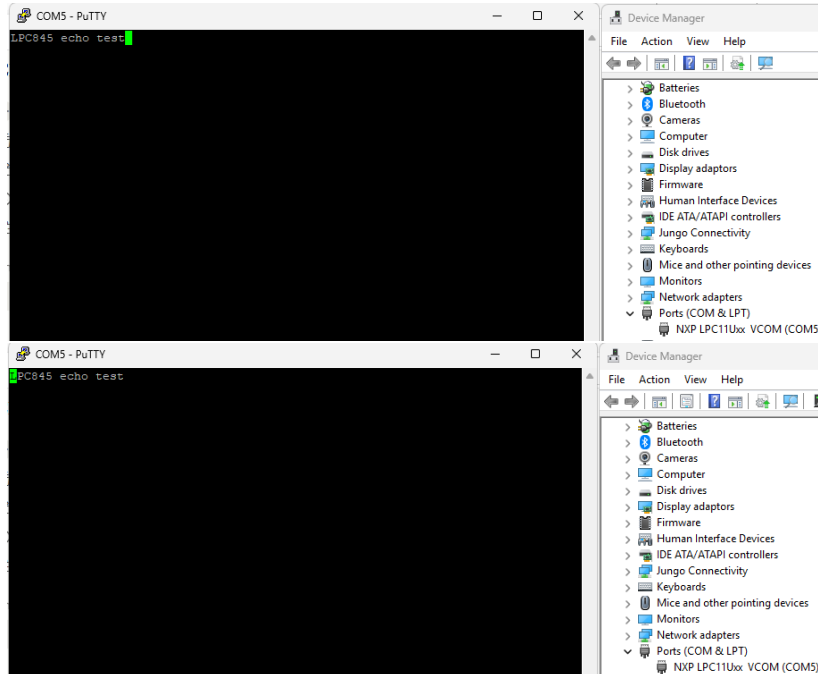


Figure 4.3 – LPC845: echo over the VCOM port exposed by the board’s own LPC11U35 debug probe

USART0 was routed onto PI00_24 (RXD) and PI00_25 (TXD) at 115 200 8N1. These are exactly the pins the on-board LPC11U35 debug probe bridges to the host as a VCOM port [14], so unlike the ESP32-C6 and RP2350 demos this one needs no external USB-to-serial adapter. Every typed character is echoed back, matching the other two platforms’ UART demos. This also gives the LPC845 a usable text-logging channel, which partly offsets the missing `defmt` integration noted above. Figure 4.3 shows a capture of this output.

The Switch Matrix step (`swm.movable_functions.u0_rxd.assign(...)`) is a good HAL-level feature and has no equivalent on the other two platforms. Because USART, SPI, I2C and several timer signals are all movable, pin conflicts that a fixed pinout table would catch on the ESP32/RP2350 instead surface here as a compile-time typestate error if one physical pin is assigned to two movable functions at once. The trade-off is one extra concept the other two platforms’ UART demos did not need.

4.3.5 ADC

Sample an analog input voltage and convert it to a digital value.

```

1  #![no_std]
2  #![no_main]
3  use panic_halt as _;
4  use cortex_m_rt::entry;
5  use lpc8xx_hal::{cortex_m::peripheral::Peripherals as CorePeripherals, prelude
    ↪ ::*, delay::Delay, gpio::Level, Peripherals};
6  use lpc8xx_hal::syscon::clock_source::AdcClock;
7  use nb::block;
8
9  #[entry]
10 fn main() -> ! {
11     let cp = CorePeripherals::take().unwrap();
12     let p = Peripherals::take().unwrap();
13
14     let mut syscon = p.SYSCON.split();
15     let mut swm = p.SWM.split();
16     let mut swm_handle = swm.handle.enable(&mut syscon.handle);
17     let gpio = p.GPIO.enable(&mut syscon.handle);
18
19     // active low: High is LED off
20     let mut led = p.pins.pio1_2.into_output_pin(gpio.tokens.pio1_2, Level::High)
    ↪ ;
21     let mut delay = Delay::new(cp.SYST);
22
23     let adc_clock = AdcClock::new_default();
24     let mut adc = p.ADC.enable(&adc_clock, &mut syscon.handle);
25
26     let (mut adc_pin, _) = swm
27         .fixed_functions
28         .adc_0
29         .assign(p.pins.pio0_7.into_swm_pin(), &mut swm_handle);
30
31     loop {
32         let value: u16 = block!(adc.read(&mut adc_pin)).unwrap();
33
34         let period_ms = 20 + (value as u32 * 500) / 4095;
35
36         led.set_low();
37         delay.delay_ms(period_ms);
38         led.set_high();
39         delay.delay_ms(period_ms);
40     }
41 }

```

Program Code 4.26 – LPC845: blocking ADC read of the on-board potentiometer on PIO0_7.

Channel 0 on `PI00_7` reads the board’s onboard potentiometer. Turning the knob visibly changed the LED’s blink rate, confirming the ADC tracks the wiper correctly. As with the RP2350, there’s no calibration step here. `lpc8xx-hal` returns the raw 12-bit SAR code with no equivalent to `esp-hal`’s `AdcCalCurve`, so converting to a voltage is left entirely to the application.

4.3.6 DAC

Generate an analog output voltage from a digital value.

Unlike the RISC-V ESP32 and the RP2350, the LPC845 does have genuine on-chip DACs: DAC0 on `PI00_17` and DAC1 on `PI00_29`, each 10 bits wide. This makes the LPC845 the only platform in this survey where a real DAC demo is possible. In practice `lpc8xx-hal` exposes no `dac` module at the version used here, so the demo below does the routing at register level, following the register sequence in the LPC84x user manual [13]: enable the IOCON, SWM and DAC0 clocks in SYSCON, power up the DAC by clearing its bit in the SYSCON PDRUNCFG register (the bits there are power-down bits, so clearing one enables the block), enable the DACOUT0 fixed-pin function by clearing its bit in the Switch Matrix PINENABLE0 register, whose bits are also active low, route `PI00_17` to the analog output by setting the `DACMODE` in that pin’s IOCON register, which also disables the pin’s digital input buffer, and then write the 10-bit code into DAC0’s `CR` register, where the `VALUE` field occupies bits 15:6. The PAC’s `value()` accessor writes into that field directly, so the listing passes the plain 0 to 1023 code with no manual shift.

```
1
2 // same as LED demo here
3 use lpc8xx_hal::pac as pac;
4
5 #[entry]
6 fn main() -> ! {
7     let cp = CorePeripherals::take().unwrap();
8     let dp = pac::Peripherals::take().unwrap();
9     let mut delay = Delay::new(cp.SYST);
10
11     // SWM is needed to enable the DACOUT0 fixed-pin function, and is not
12     // clock-gated on by default on the LPC845
13     dp.SYSCON
14         .sysahbclkctrl0
15         .modify(|_, w| w.iocon().set_bit().swm().set_bit().dac0().set_bit());
16
17     // power up the DAC: the PDRUNCFG bits are power-DOWN bits, so clear it
```

```

18     dp.SYSCON.pdruncfg.modify(|_, w| w.dac0().clear_bit());
19
20     // route the analog output to PI00_17. PINENABLE0 bits are active low:
21     // clearing the bit enables the fixed-pin function.
22     dp.SWM0.pinenable0.modify(|_, w| w.dacout0().clear_bit());
23
24     // DACMODE also disables the pin's digital input buffer
25     dp.IOCON.pio0_17.modify(|_, w| unsafe { w.dacmode().set_bit() });
26
27     let mut level: u16 = 0;
28     let mut rising = true;
29     loop {
30         dp.DAC0.cr.write(|w| unsafe { w.value().bits(level) });
31
32         if rising {
33             if level >= 1020 {
34                 rising = false;
35             } else {
36                 level += 4;
37             }
38         } else {
39             if level == 0 {
40                 rising = true;
41             } else {
42                 level -= 4;
43             }
44         }
45         delay.delay_ms(5u32);
46     }
47 }

```

Program Code 4.27 – LPC845: 10-bit DAC0 triangle wave on PI00_17, written directly against the PAC.

Ramping the 10-bit code from 0 to 1020 and back should produce a triangle wave on PI00_17 between roughly 0 V and VDD, though this was not verified on an oscilloscope. This is a true analog output rather than the RC-filtered PWM used as a substitute on the other two boards, so it needs no external components and has no residual switching ripple. The step of 4 is chosen so that the counter reaches its bounds exactly and never wraps; with a step that does not divide the range evenly, the `level -= 4` branch would underflow a `u16` and panic in a debug build.

4.3.7 PWM

Generate a pulse-width-modulated signal, for example to dim an LED.

This is the second of the two demos in this survey with no HAL path at all. `lpc8xx-hal` has no PWM/SCT driver, so this demo configures SCT0 directly through `lpc845-pac`, following the register-level pattern NXP documents for the LPC8xx SCT [15]: a unified 32-bit counter with auto-limit, one match register for the PWM period, a second for the duty point, and two events tied to those matches that set and clear the output.

```

1  #![no_std]
2  #![no_main]
3
4  use panic_halt as _;
5  use cortex_m_rt::entry;
6  use lpc8xx_hal::pac::Peripherals;
7
8  #[entry]
9  fn main() -> ! {
10     let dp = Peripherals::take().unwrap();
11
12     dp.SYSCON.sysahbclkctrl0.modify(|_, w| w.sct().set_bit().swm().set_bit());
13     dp.SWM0.pinassign7.modify(|_, w| unsafe { w.sct_out0_o().bits(33) }); //
    ↪ PIO1_1
14
15     dp.SCT0.regmode.write(|w| unsafe { w.bits(0) }); // both as MATCH
16     dp.SCT0.config.write(|w| w.unify().set_bit().autolimit_1().set_bit());
17
18     dp.SCT0.capctrl_matchrel_sctmatchrel()[0].write(|w| unsafe { w.bits(24_000)
    ↪ });
19     dp.SCT0.capctrl_matchrel_sctmatchrel()[1].write(|w| unsafe { w.bits(0) });
20
21     dp.SCT0.event[0].ctrl.write(|w| unsafe { w.matchsel().bits(0).combmode().
    ↪ bits(0b01) });
22     dp.SCT0.event[0].state.write(|w| unsafe { w.bits(1) }); // enable in
    ↪ state 0
23     dp.SCT0.event[1].ctrl.write(|w| unsafe { w.matchsel().bits(1).combmode().
    ↪ bits(0b01) });
24     dp.SCT0.event[1].state.write(|w| unsafe { w.bits(1) });
25
26     dp.SCT0.out[0].set.write(|w| unsafe { w.bits(1 << 0) });
27     dp.SCT0.out[0].clr.write(|w| unsafe { w.bits(1 << 1) });
28
29     dp.SCT0.ctrl.modify(|_, w| w.clrctr_1().set_bit().halt_1().clear_bit());
30     let mut duty: u32 = 0;
31     let mut rising = true;
32     loop {

```

```

33     dp.SCT0.capctrl_matchrel_sctmatchrel()[1].write(|w| unsafe { w.bits(duty
    ↪ ) });
34
35     if rising {
36         duty += 240;
37         if duty == 24_000 { rising = false; }
38     } else {
39         duty -= 240;
40         if duty == 0 { rising = true; }
41     }
42     cortex_m::asm::delay(300_000)
43 }
44 }

```

Program Code 4.28 – LPC845: SCT0-based PWM on PI01_1 (the blue LED), written directly against the PAC.

The blue on-board LED on SCT0_OUT0 (PI01_1) fades smoothly. This time there is no HAL, no typed `Config` struct, and register and bit-field names read directly out of the user manual [13].

Each `unsafe { w.bits(...) }` writes a raw value directly into a register field, bypassing the typed accessors `svd2rust` normally generates. It is unsafe because the crate cannot verify the value is valid for that field.

4.3.8 Summary and Assessment

LED, switch, UART and ADC are all reachable through `lpc8xx-hal`'s `embedded-hal`-implementing API. PWM and DAC have no HAL driver at all, so both the pin routing and the peripheral configuration were written directly against `lpc845-pac`.

The LPC845 is the only chip surveyed with a real on-chip DAC. The on-board CMSIS-DAP probe and VCOM port give a single-cable flash-and-observe loop that the Pico 2 W does not have. `lpc8xx-hal`'s own README disclaims API stability and therefore these listings are more likely than the ESP32 and RP2350 ones to need small adjustments against whatever version ends up pinned in `Cargo.lock`.

Crates used: `lpc8xx-hal` 0.10, which re-exports `lpc845-pac` 0.4 as `lpc8xx_hal::pac`, `cortex-m`, `cortex-m-rt`, `panic-halt` and `embedded-hal` 0.2, all from the `lpc-rs` organisation.

5 Comparative Discussion

5.1 Comparison Table

Table 5.1 summarises the assessment against the five criteria of Section 3.2, and Table 5.2 reduces the peripheral-coverage column to a per-demo matrix.

Table 5.1 – Qualitative comparison of Rust support across the three platforms.

Platform	Crate maturity	Documentation	Peripheral coverage	Toolchain, flashing and debugging
ESP32-C6 (RISC-V)	High: official <code>esp-hal</code> 1.x; GPIO/UART/SPI/I2C stable, but <code>Delay</code> , ADC, timers and LEDC all still behind the <code>unstable</code> feature flag	Good: Rust on ESP Book ([4]), docs.rs, <code>esp-generate</code> templates, community tutorials	5/6: no on-chip DAC on any RISC-V ESP32; PWM plus an RC filter as the analog-output workaround	Simple: <code>rustup</code> + <code>esp-generate</code> , single <code>cargo run</code> and live <code>defmt</code> over the built-in USB-Serial/JTAG port
RP2350 (Pico 2 W)	High: community-driven <code>embassy-rp</code> , actively developed, <code>embedded-hal</code> 1.0 and <code>async</code>	Good: Embassy Book ([5]), docs.rs and a runnable example for every peripheral used	5/6: no on-chip DAC on any RP2 chip	Mixed: build and flash are one <code>cargo run</code> via <code>picotool</code> but there is no RTT channel; project assembled by hand, <code>memory.x</code> and <code>build.rs</code> written manually
LPC845 (LPC845-BRK)	Low-medium: no official HAL; community <code>lpc8xx-hal</code> explicitly not API-stable, <code>embedded-hal</code> 0.2 only, no PWM/SCT and no DAC driver; <code>lpc845-pac</code> last released around 2021	Sparse: docs.rs API docs generated from source, a handful of repository examples, otherwise the NXP user manual and raw register access	4/6 via the HAL; DAC and PWM implemented entirely against <code>lpc845-pac</code> and the SCT/IOCON registers	Simple hardware-wise (on-board CMSIS-DAP probe and VCOM on one USB cable) but no project template: <code>memory.x</code> copied by hand, and no <code>defmt</code> integration

Table 5.2 – Per-demo coverage. **H** = implemented through the HAL, **R** = implemented against raw PAC registers, **–** = not possible (no such peripheral on the chip). The denominator counts only demos the hardware supports, so it is 5 for the ESP32-C6 and RP2350 and 6 for the LPC845

Platform	LED	Switch	UART	ADC	DAC	PWM	Total via HAL
ESP32-C6	H	H	H	H	–	H	5/5
RP2350	H	H	H	H	–	H	5/5
LPC845	H	H	H	H	R	R	4/6

5.2 Binary Size

Table 5.3 lists flash and static RAM usage for every demo, from `cargo size --release` against the crate versions pinned in `Cargo.lock`. Flash is `text` plus `data`, since

initialised variables are stored in flash and copied to RAM at startup. Static RAM is `data` plus `bss`, and excludes the stack, which `cargo size` does not report. None of the three projects enables link-time optimisation or a size-oriented `opt-level`, so these are the sizes of the configuration as committed rather than the smallest achievable ones.

Table 5.3 – Flash and static RAM usage per demo, in bytes.

Platform	Demo	Flash	Static RAM
ESP32-C6	LED	540 842	2 072
	Switch	541 328	2 072
	UART	550 692	3 180
	ADC	541 298	2 072
	PWM	543 586	2 160
RP2350	LED	295 876	20 920
	Switch	16 548	1 456
	UART	16 948	1 644
	ADC	16 816	1 480
	PWM	16 932	1 600
LPC845	LED	1 044	516
	Switch	1 064	516
	UART	6 372	516
	ADC	1 720	516
	DAC	868	4
	PWM	680	4

What each demo does barely shows up in these numbers. Every ESP32-C6 binary lands within two percent of the same 540 KB, whether it toggles a pin or runs an async UART echo, so nearly all of that is fixed `esp-hal` and `defmt`/RTT baseline.

The RP2350 LED demo is the one outlier, at roughly eighteen times the flash of the other four. That is the CYW43439 firmware blobs from Section 4.2.2, not GPIO. The switch demo drives an external LED on a plain pin and needs 16 548 bytes.

The two LPC845 demos written against `lpc845-pac` are the smallest binaries in the survey, using 4 bytes of static RAM where every `lpc8xx-hal` demo carries 516. What Section 4.3 describes as a loss in safety buys back footprint.

None of this constrains the hardware. The largest LPC845 binary uses under ten percent of the chip’s 64 KB of flash and about three percent of its 16 KB of SRAM, which is the figure that matters, since it is the most constrained of the three and the board already in the Control Lab.

5.3 Lessons Learned

The following observations stood out once all three platforms had been carried through the same six demos.

Time-to-first-blink tracked community size closely. The RP2350 (`embassy-rp`) and the ESP32-C6 (`esp-hal`) were both usable within roughly the same short setup time, both being officially or as-good-as-officially backed projects with scaffolding tools and single-command flashing. The LPC845 took noticeably longer to get even the LED demo running, purely because there was no template to start from and the SYSCON/GPIO-enable boilerplate had to be pieced together from the HAL’s own example sources.

The debugging experience depends on the board at least as much as on the crate ecosystem. `defmt-over-RTT` worked seamlessly on the ESP32-C6, whose DevKit has a USB-JTAG bridge on the chip itself. While the Pico 2 W has no debug probe at all and `picotool` is flash-only, the LPC845-BRK gets a usable substitute from its CMSIS-DAP probe and VCOM port.

The generational gap between `embedded-hal` 0.2 (`lpc8xx-hal`) and `embedded-hal` 1.0/async (`esp-hal`, `embassy-rp`) means that even the demos that work equivalently well are not drop-in portable between platforms. A driver crate written against the 1.0 traits cannot be pointed at the LPC845 without an adapter, which matters for the Control Lab if existing driver crates for sensors or actuators are expected to be reused across boards.

Dropping to the PAC reduces the hardware-level safety guarantees provided by the HAL. Once every register write goes through `unsafe { w.bits(...) }`, the compiler can no longer catch a wrong pin number, a re-asserted halt bit or an unenabled event.

5.4 Limitations of This Survey

This survey covers a single board per platform and a fixed, relatively simple set of demo peripherals. It does not assess wireless connectivity, DMA-based drivers, real-time scheduling or async I/O in depth, and its qualitative assessments reflect the experience of a single developer rather than a systematic usability study. Timing figures such as “time to first blink” are impressions rather than measurements.

Two crate-specific caveats apply. `lpc8xx-hal` explicitly does not guarantee API stability, and several of its function and field names in its own repository examples and in the documentation found online did not compile against the versions pinned in Cargo.lock. The listings in Section 4.3 are the corrected versions, checked against the pinned crates rather than against the published examples. The ESP32 ecosystem

moved during the project as well: `esp-hal-embassy` was merged into `esp-rtos` ([9]) and `esp-wifi` was renamed to `esp-radio`, so older tutorials and blog posts about ESP32 Rust will not compile unmodified against the crate versions used here.

Analog output levels were not measured. The RC-filtered PWM substitute on the ESP32-C6 and the LPC845's DAC0 both produce a voltage that requires a multimeter or an oscilloscope to observe, and neither instrument was available while this report was written. Both demos are reported as building and executing, with the resulting output levels unverified. Confirming them requires no more than a bench meter and is the smallest remaining gap in this survey.

6 Conclusion

This project set out to assess how well Rust currently supports microcontroller development in practice, using six representative peripheral demos implemented across three platforms with very different levels of official and community investment in Rust tooling.

The RISC-V ESP32 and the RP2350 benefit from an actively maintained HAL built on the modern, partly async `embedded-hal` 1.0 generation of traits, both were straightforward to bring a first demo up on within minutes, and both cover five of the six demos directly. `embassy-rp` had a slight edge in API polish, since it is part of the Embassy project itself and provides the time driver directly, while `esp-hal` needs a separate integration crate (`esp-rtos`) to work with the same executor and much of its peripheral surface, including the blocking delay, remains behind an `unstable` feature flag.

The LPC845 lags behind both at the software level. Its peripheral-access crate is complete but very low-level. The community HAL was a useful finding in itself, since the project brief assumed none existed. It covers four of six demo peripherals and explicitly disclaims API stability while two demos (DAC and PWM) had to be implemented directly against raw peripheral registers.

As a hardware target, Rust on the LPC845 is usable today for teaching the core language-level ideas using `lpc8xx-hal` for LED, switch, UART and ADC, provided that PWM and DAC are either skipped or explicitly taught as register-level exercises, and provided that students are warned that the HAL's API may shift between semesters. If the lab were choosing new hardware with the smoothest possible embedded-Rust learning curve as the primary goal, either the ESP32-C6 or the RP2350 (with a debug probe) would serve students clearly better since both let a student reach a working LED, UART and PWM demo through documented APIs, though `esp-hal`'s ADC and PWM still require the unstable feature.

References

- [1] P. Kerkour, “Introduction to Embedded Development with Rust,” kerkour.com. <https://kerkour.com/introduction-to-embedded-development-with-rust>
- [2] probe-rs - debugging and flashing toolkit for embedded Rust. <https://probe.rs>
- [3] Raspberry Pi, “picotool”. <https://github.com/raspberrypi/picotool>
- [4] Espressif Systems, “The Rust on ESP Book.” <https://docs.espressif.com/projects/rust/book/>
- [5] Embassy Project, “Embassy Book.” <https://embassy.dev/book/index.html>
- [6] lpc-rs, “lpc-pac” <https://github.com/lpc-rs/lpc-pac>; the LPC845-specific peripheral access crate is published separately at <https://crates.io/crates/lpc845-pac>.
- [7] embassy-rs, “embassy - modern embedded framework using Rust and async” (GitHub repository; see `examples/rp235x` for the RP2350 demos referenced in Section 4.2). <https://github.com/embassy-rs/embassy>
- [8] esp-rs, “esp-hal - no_std hardware abstraction layers for ESP32 microcontrollers”. <https://github.com/esp-rs/esp-hal>
- [9] esp-rs, “esp-rtos” - scheduler and Embassy executor integration for esp-hal. <https://crates.io/crates/esp-rtos>
- [10] Espressif Systems, “ESP32-C6 Technical Reference Manual,” Version 1.2, 2025. https://www.espressif.com/sites/default/files/documentation/esp32-c6_technical_reference_manual_en.pdf
- [11] lpc-rs, “lpc8xx-hal - hardware abstraction layer for LPC82x and LPC845 series”. <https://github.com/lpc-rs/lpc8xx-hal>
- [12] lpc-rs, “lpc845-pac - peripheral access crate for the NXP LPC845. <https://crates.io/crates/lpc845-pac>
- [13] NXP Semiconductors, “UM11029: LPC84x User Manual,” Rev. 1.7, 14 April 2021.
- [14] NXP Semiconductors, “UM11181: LPC845-BRK Board User Manual,” Rev. 1.1, 4 February 2019.
- [15] NXP Community, “SCT-based PWM generation on LPC8xx” (LPC Microcontrollers Knowledge Base). <https://community.nxp.com>
- [16] Raspberry Pi Ltd., “RP2350 Datasheet,” 2024.
- [17] Anthropic, “Claude” , <https://www.anthropic.com/claude>, accessed 2026.

AI Tool Usage

In my report, I used the following AI tools as described below:

1. Adjusting the structure of the report, modifying section drafts, and broadening the prose of all sections supported by Claude [17].
2. Citation formatting, reference list construction, and consistency checks across sections supported by Claude [17].
3. Debugging source codes that required bit-banging, especially in LPC845, supported by Claude [17].
4. Debugging package dependencies and crate version issues during development across all boards, supported by Claude [17].
5. LaTeX formatting, figure and table structuring, and captions supported by Claude [17].

A Repository Layout and Build Instructions

The three platforms are kept as three independent Cargo projects rather than as one workspace, because each requires a different default build target and a different linker configuration.

```
rwu-rs/  
  esp32-c6-dev-kit-nx/  
    .cargo/config.toml      # riscv32imac target + probe-rs runner  
    Cargo.toml  
    build.rs  
    src/bin/{led,switch,uart,adc,pwm}.rs # pwm binary with a filter on output  
    ↪ is used for dac  
  rp2350a/  
    .cargo/config.toml      # thumbv8m.main target + picotool runner  
    Cargo.toml  
    build.rs                # copies memory.x, adds RP235x link args  
    memory.x  
    cyw43-firmware/         # 43439A0.bin, 43439A0_clm.bin, nvram_rp2040.bin  
    src/bin/{led,switch,uart,adc,pwm}.rs  
  lpc845-brk-rev-a/  
    .cargo/config.toml      # thumbv6m target  
    Cargo.toml  
    memory.x  
    src/bin/{led,switch,uart,adc,dac,pwm}.rs
```

Program Code A.1 – Repository layout.

Each demo is built and flashed from inside its own project directory using the commands in Table 3.1.

B Pin Assignments Used in This Survey

Table B.1 – Pin assignments per demo and platform. “ext.” marks a signal that required external wiring; all other signals are on-board.

Demo	ESP32-C6-DevKit-NX	Pico 2 W (RP2350)	LPC845-BRK Rev A
LED	GPI07 (ext. LED + resistor)	CYW43439 GPIO0 (on-board LED), or PIN_15 (ext.)	PI01_2, red segment, active low
Switch	GPI09, on-board BOOT button	PIN_16 (ext. button, internal pull-up)	PI00_4, User button K3, 10 k Ω pull-up
UART	GPI00 TX, GPI01 RX (ext. adapter)	PIN_0 TX, PIN_1 RX (ext. adapter)	PI00_25 TXD, PI00_24 RXD (on-board VCOM)
ADC	GPI02, ADC1 channel 2 (ext. potentiometer)	PIN_26/27/28, ADC0–2 (ext.)	PI00_7, ADC channel 0 (on-board potentiometer RV1)
DAC	none; PWM on GPI03 + RC filter (1 k Ω , 10 μ F)	none; not implemented	PI00_17, DAC0, true 10-bit output
PWM	GPI03, LEDC channel 0	GPI015, slice 7 B; GPI04, slice 2 A	PI01_1, SCT0_OUT0, blue segment (SWM pin number 33)